Ders 12 Genişletilmiş Notu: Uygulamalar: Memory Networks'ten Metin Üretimine ve Diyalog Sistemlerine

Ders 12 Genişletilmiş Notu: Uygulamalar: Memory Networks'ten Metin Üretimine ve Diyalog Sistemlerine

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Memory Networks'ün Tekrarı
- 2. End-to-End Memory Networks
- 3. Bulunan Sonuçlara İtiraz: Veri Kümesi ve İş Ayrı Şeylerdir
- 4. QANet: Paragrafta Cevabın Başını ve Sonunu Bulmak
- 5. Seçmekten Üretmeye
- 6. Otoregresif Üretim ve Bir Sonraki Token Tahmini
- 7. LSTM ile Şiir Üretimi
- 8. Üretici Modelleri Değerlendirmek
- 9. Haber Üretimi ve Şiirle Karşılaştırma
- 10. Diyalog Sistemleri: ELIZA ve PARRY
- 11. Diyalog Derlemleri ve Soft Cosine
- 12. RNN'lerle Diyalog Öğrenme ve Microsoft Tay
- 13. Sanal Asistanlar
- Örnek Sorular
- Dersin Toparlaması

1. Memory Networks'ün Tekrarı

Ders, ders notlarının "Uygulamalar" başlıklı Bölüm 6'sından devam etti. Önceki derste soru cevaplama sistemlerine başlanmış, bAbI veri kümesi ve Memory Networks görülmüştü. Hoca önce bu yapıyı kısaca tekrar etti (00:08:23).

1.1 Bellek, soru ve yönlendirme

bAbI örneğinde hikâyenin cümleleri belleği (memory), sondaki "Süt nerede?" cümlesi soruyu oluşturur (ekrandan, 00:10:00). Slayttaki Türkçe çeviri şöyledir:

```
1 Mary banyoya gitti.
2 Sandra bahçeye gitti.
3 Daniel bahçeye gitti.
4 Daniel ofise gitti.
5 Sandra oradaki sütü aldı.
6 Sandra sütü oraya bıraktı.
7 Süt nerede?          Cevap: bahçe     Cevapla ilgili cümlelerin indisleri: 6 2
```

Memory Networks, sorunun cevabının bellekteki hangi bloklardan yararlanılarak bulunacağını seçen bir mekanizmadır. Eğitim kümesinde yalnız cevap ("bahçe") değil, cevapla ilgili

cümlelerin indisleri (6 ve 2) de verilir. Bu bilginin eğitim kümesinde olmasının sebebi, eğitim sürecinde kayıp (loss) fonksiyonunun onu kullanmasıdır.

Hoca tasarımcıların niyetini şöyle anlattı: makine cevabı önümüze getirsin, ama getirirken biraz yönlendirilsin. "Cevabı bul da nasıl bulursan bul" denmez; "önce bellekte bununla ilgili, işine yarayacak parçaları bul, sonra onları kullanarak ilerle" denir.

1.2 İki adımlı çıkarım

Slayttaki tanımlar (ekrandan, 00:11:00):

- YSA + bellek mimarisi (yalnız yazıyor ve okuyor, değiştirme yok).
- m: hikâyenin tutulduğu bellek.
- m_{o_1} ve m_{o_2}: belleğin cevabı üretecek parçaları.
- x: soru; r: tek kelimelik cevap.
- s_O ve s_R'nin U'ları farklıdır. U matrisleri eğitim verisinden öğrenilir; U, $n \times d$'lik bir matristir.
- Teta (Φ) fonksiyonu metni $d \times 1$ boyutlu bir gösterime çevirir (BoW).

$$o_1 = O_1(x, \mathbf{m}) = \arg \max_{i=1,\ldots,N} s_O(x, \mathbf{m}_i)$$

$$o_2 = O_2(x, \mathbf{m}) = \arg \max_{i=1,\ldots,N} s_O([x, \mathbf{m}_{o_1}], \mathbf{m}_i)$$

$$r = \arg \max_{w \in W} s_R([x, \mathbf{m}_{o_1}, \mathbf{m}_{o_2}], w)$$

$$s(x, y) = \Phi_x(x)^\top U^\top U \Phi_y(y)$$

İlk adımda soru x ile her bellek bloğu m_i arasındaki benzerlik hesaplanır ve en benzeyen seçilir; o_1 bir indistir. İkinci adımda soru ile birinci seçilen blok birlikte kullanılarak, cevabı bulmada kullanılacak ikinci bellek bloğu aranır; sonuç o_2 indisidir. Son adımda soru, birinci ve ikinci ilgili blok birleştirilir (concatenation) ve bu kez W kümesindeki en benzer eleman cevap olarak döner. W bellek bloklarından değil, sözlükten (vocabulary) oluşur. Hoca, bu makalede çözülen problemlerde sözlüğün oldukça küçük, yüz civarında kelime olduğunu hatırlattı. s_O ve s_R iki farklı benzerlik işlemidir ve U matrisleri farklıdır.

1.3 Kayıp fonksiyonu

Kayıp fonksiyonunda m_{o_1} diye gösterilen kısım örnekteki 6. cümledir (00:13:29). Model, soru ile 6. cümle arasındaki benzerliğin yüksek, 6 dışındaki her şeyle benzerliğin düşük olmasını ister. İkinci basamakta 2. cümleyle benzerliğin yüksek, diğerleriyle düşük olması istenir. Son basamakta gerçek cevapla benzerlik yüksek, sözlükteki diğer bütün kelimelerle düşük olmalıdır. Formülde üstü çizgili gösterilen her şey gerçek cevabın (6, 2 ve "bahçe") dışındaki seçeneklerdir.

Özetle Memory Networks, bellekte soruyla ilgili kısımları bulup onlardan yararlanmak için içsel mekanizmayı bir miktar yönlendirmek amacıyla tasarlanmış mimarilerdir.

2. End-to-End Memory Networks

2.1 Destekleyici indislerden kurtulmak

Bunun gelişmiş hali End-to-End Memory Networks'tür (00:14:56). Buradaki "end-to-end" ile kastedilen, eğitim kümesindeki 6 ve 2 bilgisine olan ihtiyacı kaldırmaktır: bu indisleri kullanmadan eğitim yapılabilir mi?

Belleğin iki temsili vardır; A temsili ve C temsili. Bellekteki cümleler (sentences) iki ayrı gömme ile temsil edilir. Soru (question) da bir gömme ile (u) temsil edilir. Memory Networks'te

en benzeyeni alıp kullanan bir yapı vardı: o_1 bulunur, sonra o_1 kullanılarak o_2 bulunurdu. Burada "budur" diye tek-sıcak (one-hot) bir seçim yapılmaz. Benzerlikler oranında ağırlıklar (p_i) hesaplanır; p_i, sorunun eldeki i. cümleye ne oranda benzediğini ifade eder.

Slayttaki formüller (ekrandan, 00:16:00):

$$o = \sum_i p_i c_i \qquad \hat{a} = \text{Softmax}(W(o + u))$$

Eğitim sürecinde A, B, C ve W gömme matrisleri öğrenilir. p_i'ler belleğin diğer temsiliyle (C) çarpılıp toplanarak ağırlıklı toplam (weighted sum) olarak bir o gömmesi bulunur. Bu o, sorunun gömmesiyle birlikte kullanılarak en benzeyen cevap bulunmaya çalışılır.

2.2 Dikkat mekanizmasıyla benzerlik

Hoca bu yapının dikkat çekici bir unsurunu vurguladı: Transformer'lardaki dikkat (attention) mekanizmasını bir miktar andırır (00:16:58). İki temsil vardır; bunları Transformer'daki anahtar (key) ve değer (value) gibi düşünebilirsiniz. En benzerler bulunur, benzerlik ağırlıklarıyla temsiller çarpılır ve "sorunun gerçek temsili budur" gibi bir sonuç çıkar. Burada soru ve cevap vardır, ama Transformer terimlerine çevirdiğinizde bunu self-attention gibi düşünebilirsiniz: bir kelimenin gerçek anlamı, çevresindeki diğer kelimelerin vektörlerinin ağırlıklandırılmış ortalaması gibi çıkar. Böylece bir kelime çevresindeki kelimelerden anlam kazanır.

2.3 Çok adımlı mimari

Bu işlemin tek adımı yerine iki ya da üç adımı yapılabilir (00:18:14). Memory Networks'te adım sayısı sabit olarak ikiydi. Slaytta solda tek adımlı, sağda üç adımlı mimari vardır (ekrandan, 00:16:00). Temsiller katmandan katmana farklılaşır: birinci katmanda A^1, C^1, ikinci katmanda A^2, C^2, üçüncü katmanda A^3, C^3. Hoca bunu bir derinleşme olarak yorumladı: her katmanda soruya göre bellekteki cümlelerin anlamı, daha doğrusu etkinliği değişir.

2.4 Konum gömmeleri

Kelimelerin bulundukları konuma göre anlam kazanması için konum gömmeleri (position embeddings, PE) tanımlanmıştır (ekrandan, 00:20:00). BoW gösterimi kelimelerin sırasını önemsemez; buna bir çözüm:

$$m_i = \sum_j l_j \cdot Ax_{ij}$$

$$l_{kj} = (1 - j/J) - (k/d)(1 - 2j/J)$$

Buradaki "$\cdot$" operatörü element bazlı çarpmadır. $d = 128$, $J = 25$ için L matrisinin her sütunu farklı bir indeksi (kaçıncı kelime olduğunu) ifade eder. Kelimenin kendisi 128 boyutludur; buna 128 boyutlu bir konum gömmesi eşlik eder. Bu konum gömmesi öğrenilmez, sabittir ve slayttaki fonksiyonla hesaplanır. Bu sayede aynı kelimenin cümleye etkisi farklı indislerde farklı olur.

Fonksiyonun özelliği şudur: kelimeler ardışık ise konum gömmeleri birbirine yakındır. Slayttaki ısı haritasında ardışık sütunlar arasında büyük bir değişiklik yoktur; birbirinden uzak sütunlar arasında ise büyük fark vardır. Böylece kelimelerin yerleri de işin içine katılmış olur.

> **Ders sonrası not:** Slayttaki formülle $d = 128$, $J = 25$ için L python ile hesaplandı: ardışık iki sütun arasındaki ortalama mutlak fark 0.02, 1. ile 25. sütun arasındaki fark 0.48 çıktı. Bu, hocanın "ardışık sütunlar yakın, uzaklar farklı" gözlemini doğrular.

Hoca bütün bunları Transformer ve BERT yapılarına doğru giden adımlar olarak algılayabileceğimizi söyledi (00:20:23).

3. Bulunan Sonuçlara İtiraz: Veri Kümesi ve İş Ayrı Şeylerdir

3.1 Üzerinde çalışılan 20 görev

End-to-end mimarilerle yapılan çalışmalarda 20 görev tanımlanmıştır; hepsi çok küçük sözlüklü bAbI görevleridir (00:20:38). Slaytta her görev için hikâye, destekleyici cümleler (Support) ve her adımda (Hop 1, 2, 3) hangi cümleye ne kadar ağırlık verildiği tablolar halinde gösterilir (ekrandan, 00:20:45). Örneğin "1 supporting fact" görevinde hikâye "Daniel went to the bathroom. Mary travelled to the hallway. John went to the bedroom. John travelled to the bathroom. Mary went to the office. Sandra journeyed to the kitchen." biçimindedir; soru "Where is John?", cevap "bathroom", tahmin "bathroom"dur. Destekleyici cümle "John travelled to the bathroom" satırıdır ve bu satır her üç adımda da en yüksek ağırlığı alır (0.60, 0.98, 0.96). Diğer görevler arasında iki ve üç destekleyici cümle, iki ve üç argümanlı ilişkiler, evet/hayır soruları, sayma (counting) ve listeler/kümeler vardır.

3.2 Anlamlı veri kümesi sorunu

Hoca burada başka bir yönün karşımıza çıktığını söyledi: eğitim kümelerinin anlamlı olup olmaması (00:20:50). Bir soru cevaplama veri kümesinde bir sistem başarılı olduysa artık bu sistemin soru cevaplama yapabildiğini söylemek doğru olmayabilir. Adı X olan bir veri kümesinde başarılı bir makine için "makineler artık X işini yapabiliyor" demek doğru bir hareket değildir, çünkü veri kümesinin içinde işin aklımıza gelmeyen ve istemediğimiz bir yöntemle çözülebildiği durumlar olabilir. Hoca bunu önceki derste anlattığı tank örneğine bağladı (00:22:02).

Slayttaki itiraz (arXiv:1808.04926v1) (ekrandan, 00:22:40):

- (metin, soru, cevap) üçlüleri genel yapıdır.
- 5 veri kümesi üzerinde 3 modelin performansı ölçülmüştür.
- Veri kümelerinin orijinal halleri, yalnız metin kısımları ve yalnız soru kısımları kullanılmıştır. Bazı veri kümelerinde (özellikle "harika sonuçlar" dediğimiz yerlerde) bu eksik hallerle orijinal veri kümesine yakın sonuçlar alınmıştır.
- Bunun anlamı, bazı veri kümeleri için sistemin yalnız metne ya da yalnız soruya bakıp cevabı tahmin edebilmesidir.

Yani bazı görevlerde soruyu sormadan yalnız metni verdiğinizde ya da metni vermeden yalnız soruyu verdiğinizde sistem cevabı bulabiliyor. Buradan, sistemin sorudan cevaba ya da metinden cevaba bizim hiç düşünmediğimiz ve istemediğimiz eşlemeler (mapping) yaptığını anlıyoruz. Biz "önce şu belleği yakala, sonra onunla birleşip ötekine git" ya da "her biriyle ağırlığını bul, soruyu ve belleği kullanarak adım adım ilerle" diye yönlendirmeye çalışsak da, istemediğimiz ve aklımıza gelmeyecek çözümler ortaya çıkabilmektedir.

> **Hocanın vurgusu:** Adı X olan veri kümesinde başarı sağlandığında "X olayı makine tarafından çözülmüştür" demek çok doğru değildir.

4. QANet: Paragrafta Cevabın Başını ve Sonunu Bulmak

4.1 Mimari

Bir başka soru cevaplama yapısı QANet'tir (ICLR 2018) (00:24:15). Hoca bunun bambaşka bir şey olmadığını, yine benzerlikler üzerine kurulu olduğunu söyledi. Bağlam (context) paragrafımız, sorgu (query) sorumuzdur; ikisi de gömmelerden geçer, sonra "context-query attention" denen bir yapıya girer. Artık dikkat (attention) burada adıyla geçmeye başlar.

Slayttaki tanımlar (ekrandan, 00:24:45):

- Paragraf (C) n, soru (Q) m kelimeden oluşsun.
- Paragrafın i. kelimesinin (c), sorunun j. kelimesine (q) benzerliği $S_{ij} = W_0[c; q; c \odot q]$.
- W_0: $3d$ boyutlu öğrenilen bir vektör. S: $n \times m$ boyutlu bir matris.
- $\bar{S}$: softmax ile satır normalizasyonu (S); $\bar{\bar{S}}$: softmax ile sütun normalizasyonu (S).
- $A = \bar{S} \cdot Q^T$, $B = \bar{S} \cdot \bar{\bar{S}}^T \cdot C^T$; Q: $d \times m$, C: $d \times n$.

Mimari şemasında her kelime, kelime bazlı (GloVe) 300 ve karakter bazlı 200, toplam 500 boyutlu gömülür. Gömme kodlayıcı bloklarından (Stacked Embedding Encoder Blocks) sonra

her kelime $d = 128$ boyutlu olur. Context-Query Attention çıkışı $4d \times n$ boyutlu $[C; A; C \odot A; C \odot B]$'dir. Bunun üzerinde üç kez model kodlayıcı blokları (Stacked Model Encoder Blocks) uygulanarak M_0, M_1, M_2 elde edilir. Sol kol başlangıç olasılığını (Concat → Linear → Softmax → Start Probability), sağ kol bitiş olasılığını (End Probability) verir.

4.2 Başlangıç ve bitiş olasılıkları

Bu modelde cevap bir kelime değildir (00:27:16). Bulunmak istenen şey, paragrafın içinde cevabın nerede başlayıp nerede bittiğidir. Paragraftaki kelimeler ya da tokenlar uzunluğunda bir vektör düşünün; her birinin başlangıç tokenı olma ve bitiş tokenı olma olasılıkları hesaplanır (ekrandan, 00:30:00):

$$p^1 = \text{softmax}(W_1[M_0; M_1]) \qquad p^2 = \text{softmax}(W_2[M_0; M_2])$$

W_1 ve W_2 slayta göre $1 \times 8d$ boyutlu öğrenilen vektörlerdir. Minimize edilen fonksiyon:

$$L(\theta) = -\frac{1}{N}\sum_{i}^{N}\left[\log\left(p^1_{y^1_i}\right) + \log\left(p^2_{y^2_i}\right)\right]$$

Burada $p^1_{y^1_i}$, i. örneğin gerçek cevabının paragrafta başladığı indisin olasılığıdır; θ öğrenilen tüm değişkenlerdir. Test zamanında cevap olarak $s \leq e$ olan ve $p^1_s p^2_e$ değeri en büyük olan (s, e) ikilisi bulunur.

Hoca dikkat çekti: bu bir üretici model değildir (00:28:22). Sözlükten bir kelime seçme yöntemi yerine, paragraftaki belli bir parçanın sorunun cevabı olduğu kabul edilir. Bu SQuAD veri kümesinin, en azından 1. sürümünün yapısıdır; 2. sürümde cevabın olmadığı sorular da vardır, ama 1. sürümde cevap illa paragrafın içindeki bir kısımdır.

4.3 Yapay veri artırımı

Bu çalışmada bir tık farklı olabilecek şey yapay veri artırımıdır (00:29:20): çift çeviri (back translation). Veri İngilizceden Fransızcaya, sonra Fransızcadan İngilizceye çevrilerek artırılır.

4.4 Attention'ın kökeni

Hoca attention kavramının doğal dil işleme alanından çıkmadığını vurguladı (00:25:04). Attention "dikkat" demektir; görüntü işleme tarafında da doğal dil işlemede de buna dair birçok çalışma vardır. Transformer'ların getirdiği farklı bakış açısı self-attention kısmıdır ve bunun öz denetimli (self-supervised) görevlerde çok büyük ölçeklerde uygulanmasıdır.

Hocaya göre bu tür yenilikler bir anda ortaya çıkmaz; ön hazırlıkları belli ölçüde görülebilir. Ama bu biraz bilerek geçmişi yorumlamaya denk gelir: bugün geldiğimiz noktayı bildiğimiz için geçmişteki ayak izlerini görebiliyoruz. İçinde bulunulan anda başarılı olacak yönü tayin etmek o kadar kolay değildir; geçmişe bakınca her şey anlamlı görünür. O anda doğru yöne gitmek bir miktar şans, bir miktar da öngörü ve vizyon gerektirir.

5. Seçmekten Üretmeye

QANet gibi yapılarda bir seçim yapılır: "cevap şudur, şuradadır" (00:29:50). BERT'in ince ayar (fine-tuning) yapılarına bakıldığında da şıklar arasından seçme ya da paragrafta cevabın başlangıç ve bitişini bulma gibi görevlere uyarlandığı görülür.

Daha sonra, 2022 aralığında ChatGPT tarafıyla (ve onun öncesiyle) yöntem değişmeye başladı (00:30:25): üretici süreçler işin içine girdi. Artık "cevap şuradan başlıyor, şurada bitiyor" ya da "kümenin içinden seç" gibi bir yapı yoktur; tamamen üretime dayalı olarak "bu sorunun cevabı ne olur?" diye düşünülür. Paragrafı verip "bu paragraftaki sorunun cevabını söyle" dersiniz ve paragraftaki bir parçayı birebir kopyalamasını beklemezsiniz. Hocaya göre insanların yaptığı da kopyalama değil, bir tür üretimdir.

Bu işteki büyük oyuncuların hepsi üretici modellere yönelmiş durumdadır (00:31:43). Tahmin eden, sınıflandıran modeller biraz arka planda kalmıştır. Şu anda çıkan büyük modellerin hepsi otoregresif üretici süreçler üzerinden hareket eder.

6. Otoregresif Üretim ve Bir Sonraki Token Tahmini

6.1 Otoregresif model

Hocanın "üretim" derken kastettiği otoregresif modellerdir (00:33:02). Otoregresif modelin anlamı bir sonraki adımı tahmin etmektir. Elinizde bir süreç vardır; belli bir yere kadarki kısmı size verilir, siz bir sonraki adımı tahmin edersiniz. Yalnız bir sonraki adımı tahmin ederek uzun bir şeyi nasıl üretiriz? Kendi ürettiğinizi kendinize girdi olarak verip devam edersiniz: x_1, x_2 verilince x_3'ü tahmin edersiniz; sonra x_1, x_2, x_3 verilmiş gibi düşünüp x_4'ü tahmin edersiniz. Böylece size bir x_1 verildiğinde devamını üretmeyi sürdürebilirsiniz. Bütün hikâye şudur: verilen bir dizinin bir sonraki tokenını tahmin edebiliyorsanız bir süreç üretebilirsiniz.

Bu eski bir hikâyedir. Hoca, yanlış hatırlamıyorsa 1980'lerin sonundaki geri yayılım (backpropagation) makalelerinde bile bir sonraki kelime tahmini (next word prediction) problemleri üzerinde uğraşıldığını söyledi (00:34:27).

6.2 Neden bu kadar etkili?

Hoca, bir sonraki token tahmininin neden bu kadar etkili olduğunu şöyle açıkladı (00:35:14): bir şeyin bir sonrasını tahmin edebilme kabiliyetiniz varsa o mevzuyu büyük ölçüde kavramışsınızdır. Hipotez budur: bir dizinin devamını tahmin edebiliyorsanız o diziyi üreten süreci kavramışsınızdır.

Ancak "kavramak" ifadesini insan gibi algılamak şart değildir. Hoca, ders başındaki örnekleri hatırlattı: bir problemi çözebiliyor olmak, o problemi bizim istediğimiz yoldan çözüyor olmak demek değildir. İnsanlara atfettiğimiz terimleri makinelere atfetmek uygun değildir; belki ikisi farklı terimlerle anılmalıdır, ama böyle bir ayrım yapmak pek kimsenin işine gelmemektedir.

6.3 Bir sınıflandırma problemi

Bir sonraki token tahmini aslında bir sınıflandırma problemidir (00:37:26). Bir girdiye karşılık sınıflardan birini söylersiniz; tahmin edilen token bir sınıftır. Şiir örneğinde 84 karakter vardır; tokenizer çıkışları kullanılsaydı 32 binlik ya da 48 binlik bir sınıf kümesi olurdu. Yapı değişmez: indis tahmin edersiniz, regresyon değil sınıflandırma yaparsınız.

Bu problemi bildiğimiz problemlerden farklı kılan eğitim kümesidir (00:38:22). Öz denetimli (self-supervised) yapıda insan müdahalesine gerek olmadan çok büyük miktarda veri kümesi üretilebilir. Bir metni bu yapıyla bir eğitim kümesine dönüştürmek çok basit bir kodla yapılır; bir sonraki karakteri ya da kelimeyi tahmin etmek için hiç insan emeği gerekmez.

7. LSTM ile Şiir Üretimi

Hoca, dersin sayfasında da bulunan bir şiir üretimi örneği gösterdi (ekrandan, 00:36:45):

- LSTM ile şiir üretimi. Kod: `siir_uretimi.m`, veri: `tum_siirler.txt`.
- Yaklaşık 4 milyon karakter içeren şiirler.
- LSTM bir sonraki karakteri tahmin etmesi için eğitildi (many to many).
- 100'den fazla geçen 84 karakter kullanıldı.
- Her bir giriş 84 boyutlu one-hot vektör.
- Her bir çıkış 84 sınıftan biri.

Burada kelime değil, yalnız karakter tahmin edilir. Model LSTM'dir; Transformer yoktur. Bütün veride yüzden fazla geçen karakterler seçilmiştir, çünkü çok az geçen garip karakterlerle uğraşmaktansa onları dışarıda bırakmak tercih edilmiştir. Girişte kelime gömmesi gibi anlamlı bir gömme yoktur; yalnız karakterin one-hot vektörü vardır.

Eğitim ayrıntıları (ekrandan, 00:40:00): ilk 200 bin karakter eğitim için kullanıldı; 512 LSTM hücresi, 500 iterasyon, yaklaşık 2 saat eğitim. Farklı başlangıçlarla üretilen örnek şiirlerden biri:

```
Kaç yolcu bu yolda yürekler
  Bir gün karanlığın beni
  Görmeyen bir gün kalmadım

  Doğmaya bakarsan bir zaman
  Saçların kış günük değil kaç yolcu
  Yaşamak bir değil
  Bir güzel ülküdür gönül verdiğim.

  Bahçem yaralı kapılar gibi
  Garip gitme kalan y'
```

Slaytın altında şu not vardır: “Falcı 10 şey söyler 1’i tutar, falcı bildi olur.”

Hocaya göre iki saatlik eğitim ve 200 bin karakterlik veri için sonuçlar fena değildir; Transformer gibi gelişmiş mimariler olmadan da fena olmayan sonuçlar alınabilir. Ama “fena olmayan” öznel bir ifadedir.

8. Üretici Modelleri Değerlendirmek

Üretici modellerin en büyük problemi değerlendirilmeleridir (00:40:26). Üretici bir şey yaptırmaya çalıştığınız için üretilmesi gerekenin ne olduğunu tam olarak söyleyemeyebilir ya da söylemek istemeyebilirsiniz. Söylediğiniz durumda da model söylediğinize çok benzeyen ama birebir eşleşmeyen (match olmayan) bir şey, aynı şeyi ifade eden başka bir şey üretebilir ve bunu algılayamayabilirsiniz. Bu yüzden bu tarz sistemlerin değerlendirilmesindeki en büyük problemlerden biri ölçeklenebilir değerlendirme sistemlerinin kurulmasıdır.

Hocaya göre bu, oldukça şık bir yolla, pekiştirmeli öğrenme (reinforcement learning) yapısıyla çözüldü ve şu anda bir standart gibi gidiyor (00:41:40); bunu ilerleyen derslerde konuşacaklarını söyledi.

> **Ek bilgi (derste anlatılmadı):** Hocanın değindiği yaklaşım büyük olasılıkla insan geri bildirimiyle pekiştirmeli öğrenmedir (RLHF). Bu yöntemde insanlar modelin farklı çıktılarını sıralar, bu sıralamalardan bir ödül modeli eğitilir ve dil modeli bu ödül modelini en büyükleyecek biçimde pekiştirmeli öğrenmeyle ayarlanır. Böylece her çıktı için tek bir “doğru cevap” yazmak gerekmez.

Gösterilen şiirlerin hiçbiri bütün olarak iyi değildir (00:42:22), ama içlerinde iyi parçalar vardır. Lokal olarak iyi görünen parçaların bulunması kolay, bütünün iyi olması zordur. Üretilen şeyin tamamının bir bütünlüğe sahip olması daha zor bir problemdir. Yine de bu çok basit bir mimari ve çok az bir veri kümesidir.

Günümüzde iddia edilen şey şudur (00:43:20): mimari ve veri kümesi birlikte büyütüldüğünde makinenin yeteneklerinin bir sınırı yoktur. Hoca “datayı” değil “iyi datayı” dediğini vurguladı: iyi veri ve model büyütüldükçe bir yerde doyup durma gibi bir durumun olmadığına dair bir hipotez vardır.

9. Haber Üretimi ve Şiirle Karşılaştırma

9.1 Haber üretimi

Aradan sonra hoca aynı tekniği haber üretiminde gösterdi (ekrandan, 01:01:20):

- Benzer teknik.
- 500 bin karakterlik spor haberleri. Kod: `haber_uretimi.m`, veri: `haber42bin.mat`.
- 512 hücre, 30 iterasyon.
- 2 saat eğitim.
- Farklı başlangıçlarla üretilen örnek haberler.

Veri kümesinde çok daha fazla haber vardır, ama yalnız 500 bin karakterlik spor haberleri kısmı alınmıştır. Slayttaki örneklerde başlangıç kelimesi verilmiş, model devamını karakter karakter üretmiştir:

```
gazete
cilere yaptığı açıklamada, türkiye futbol federasyonu (tff) tarafından düzenlenen
 taraftarların ardından başkan aziz yıldırım ile birlikte yapılan açı

defans
oyuncusu burak yılmaz vural, "bu sezon iyi bir takım olarak görevini yaptı.
bu kadar değil. bu kadar değil. bu kadar değil. bu kadar değil. bu kadar

cimbom
tarafından da tek tek kaybetti ancak bu kararın ardından bu sezon ilk kez
karşılaşacak olan ispanya ligi ile başlayan bir takım olarak görev yapan t
```

“gazete” başlangıcı “gazetecilere” diye, “defans” başlangıcı “defans oyuncusu” diye devam etmiştir. Ardışık parçalar lokal olarak iyi görünür (“gazetecilere yaptığı açıklamada”, “Türkiye Futbol Federasyonu tarafından düzenlenen”). “Defans oyuncusu Burak Yılmaz” deyip sonra “Yılmaz Vural”a kayması gibi geçişler görülür.

9.2 İstatistiksel dil modelleriyle bağlantı

Hoca bunu daha önce konuşulan istatistiksel dil modellerine bağladı (01:01:58). Orada bir pencere vardı; iki öncesi, iki sonrası gibi bir pencerenin içindekiler işleme alınırdı ve pencere büyüdükçe daha büyük birliktelikler anlamlı hale gelirdi. Onların problemi, lokallerin anlamlı ama bütünün anlamlı olmayışıydı. Burada da benzer bir problem görülür.

Transformer’lar bunu bir miktar daha çözer (01:02:54): veriyi adım adım değil, paralel olarak hepsini birden aldıkları için en azından bağlam (context) uzunluğu kadar geçmişi işin içine katabilirler. Üretilen haberlerde lokal olarak iyi yerler vardır ama bütüne bakıldığında bir haber üretildiğini söylemek güçtür. Yine de çok küçük bir veri kümesi ve küçük bir mimariyle bu yapılar ortaya çıkabilmektedir; veri ve model büyüdükçe daha başarılı olacağı tahmin edilebilir.

Hoca, büyük modellerin buradan farkının Transformer kullanması ve her yönde büyüklük olduğunu, kayıp fonksiyonu açısından bir fark olmadığını söyledi (01:04:07): kayıp tamamen bir sonraki token tahminidir (next token prediction). O modelin ürettiklerine bakıldığında bütünlüğün de bir ölçüde sağlandığını, bir insan okuduğunda gariplik sezmediğini ifade etti.

9.3 Şiir mi, haber mi?

Hoca şiir ve haber üretimini karşılaştırdı (01:05:11). Şiirler biraz daha “hata kaldırır”. Şiirde bir ifadeyi tam anlamasanız da şairin sembolik bir ifade kullandığını düşünebilirsiniz; şiir üretiminde yapılan garip şeyleri yorumlayabilirsiniz. Düz metin öyle değildir: üretim sürecindeki bir hata insanların gözüne doğrudan takılır. Örneğin “Yaşamak bir değil” normalde garip gelir, ama anlamlı bir şiirin içinde olabilir mi? Olabilir. “Bir güzel ülküdür gönül verdiğim” fena değildir. “Bahçem yaralı kapılar gibi”deki “yaralı kapı” yorumlanabilir. Düz metinde böyle değildir. Mimari açısından iki örnek arasında fark yoktur.

Hoca bir farkı daha atlamamak gerektiğini söyledi: çeşitlilik (01:07:25). Şiirler çok daha çeşitlidir (diverse); farklı konularda, farklı yapılardadır. Herhangi iki şiirin birbirine benzerliği çok yüksek değildir. Herhangi iki spor haberi arasında ise epey benzerlik bulursunuz; spor haberlerinde belli bir yapı vardır, küçük şeyler değişir. Veri büyüklüğü açısından büyük fark olmasa da şiir tarafındaki yüksek çeşitlilik, şiir üretiminin bir tık daha başarılı değerlendirilmesini sağlar. Bu örnek çeşitliliğin önemini de ortaya çıkarır.

10. Diyalog Sistemleri: ELIZA ve PARRY

10.1 İki tür

Sonra diyalog sistemlerine geçildi (ekrandan, 01:10:00). Slayttaki “Sohbet ve sanal asistan uygulamaları”:

- Sadece sohbet amaçlı: gerçeklemek çok zor.
- Hedefi olan: ürün/hizmet bulan, alan; kısıtlı bir uzay, daha kolay.

Sadece sohbet amaçlı sistemler, bir arkadaşınızla yaptığınız muhabbeti makineye yaptırma arayışıdır. Bunlar gerçekten zordur, çünkü çeşitlilik çok daha yüksektir. Bir banka ya da ticaret uygulamasının sanal botuyla yapabileceğiniz konuşmaların çeşitliliğini, bir de arkadaşınızla yapabileceğiniz konuşmaların çeşitliliğini düşünün. Hoca öznel bir görüş olarak, farklı farklı arkadaşlarınızla çeşitli şeyler konuşabiliyorsanız bunun sizin için iyi olduğunu ekledi. Ticari anlamda işe yarayan kısım hedefi olan yapılardır ve bu problemler sohbet amaçlılara göre daha kolaydır.

10.2 ELIZA

Diyalog sistemleri ELIZA ile başlar (1965, Joseph Weizenbaum); ELIZA bir terapisti simüle eder (ekrandan, 01:10:50). Slayttaki diyalog örneği (kullanıcının sözleri normal yazımla, ELIZA'nın cevapları büyük harfle):

```
Men are all alike.
IN WHAT WAY?
They're always bugging us about something or other.
CAN YOU THINK OF A SPECIFIC EXAMPLE?
Well, my boyfriend made me come here.
YOUR BOYFRIEND MADE YOU COME HERE?
He says I'm depressed much of the time.
I AM SORRY TO HEAR THAT YOU ARE DEPRESSED.
It's true that I am unhappy.
DO YOU THINK COMING HERE WILL HELP YOU NOT TO BE UNHAPPY?
I need some help, that much seems certain.
TELL ME MORE ABOUT YOUR FAMILY.
My mother takes care of me.
WHO ELSE IN YOUR FAMILY TAKES CARE OF YOU?
My father.
YOUR FATHER
```

ELIZA'nın çok basit, şablon tabanlı bir yapısı vardır; parametrik dönüşüm tabloları kullanır: "şöyle bir şey geldiyse şuna dönüştür". "ELIZA nasıl çalışır?" slaytı (ekrandan, 01:12:10): ELIZA'nın çalışma mekanizmasına bakıldığında tüm etkileyiciliği biraz kaybolur.

```
Eğer uyan şablon varsa dönüşüm uygula ve cevap ver:
  (X me Y)                 ---> (X you Y)
  (I remember X)           ---> (Why do you remember X just now?)
  (My {family-member} is Y) ---> (Who else in your family is Y?)
  (X {family-member} Y)    ---> (Tell me more about your family)
Yoksa genel amaçlı cevaplarından birini ver.
```

Arka tarafta işlerin nasıl döndüğünü bilmeyen bazı insanlara 1965'te ilginç gelmiştir. Turing testinde zeki olmadığı açıktır, ama az da olsa kandırabildiği insanlar olmuştur. ELIZA çoğunlukla kullanıcı girişini dönüştürerek kullanıcıya geri söyler (01:11:20). Bu, Rogerian psikoterapi tekniğidir: bir kişi kendi sözlerini başkasının ağzından duyduğunda o söylediği şeye dair farkındalığı artar, onun üzerine bir daha düşünür.

10.3 PARRY

1965'teki ELIZA'dan sonra 1971'de PARRY (Colby) gelir (01:11:47). ELIZA terapisti taklit ederken PARRY hastayı, paranoyak bir hastayı taklit eder. Bu sistemler korkunç emek isteyen işler değildir, ama PARRY için epey uğraşılmıştır; yine hep şablon tabanlıdır.

Hoca paranoyak düşünce biçimini şöyle anlattı: paranoyaklar olmayan tehditleri algılar; sürekli tehdit algılamaya yöneliktirler. Bu bir şekilde kendini önemli hissetmeyle ilgilidir: "Bütün Amerika, CIA peşimde; demek ki ben çok önemli birisiyim." PARRY'de karşı taraf bir şey söylediğinde onun amacının ne olduğunu, ne demek istediğini okumaya çalışan, ama bunu kötücül bir biçimde yapan bir akıl okuma temsil edilmeye çalışılmıştır. Bunun üzerine epey kafa yorulmuştur, ama tamamen elle kodlama denebilecek yapılarla ilerlenmiştir.

10.4 Elle kodlamanın bedeli

Böyle bir sistemi gerçekleştirmenin yollarından biri elle kodlamaktır (01:13:30), ama bunun maliyeti hiç düşük değildir ve uyarlanabilirliği çok düşüktür. A sistemi için harcadığınız emeği B sistemine kolay aktaramazsınız.

11. Diyalog Derlemleri ve Soft Cosine

11.1 En benzer ifadeyi bulmak

Daha ölçeklenebilir yapılar kurmak istediğimizde karşımıza çıkan en basit yaklaşım diyalog derlemlerini kullanmaktır (01:14:03). Derlem, "kullanıcı şunu dediğinde sen şunu de" biçiminde, soru-cevap gibi bir yapıdır. Böyle diyaloglar pek çok yerden toplanabilir: filmler, diziler ve bunlardaki diyaloglar; kendi ya da şirketinizin WhatsApp diyalogları gibi.

Slayttaki "Diyalog derlemleri kullanalım" (ekrandan, 01:14:40):

- Kuralları oluşturmak ve yönetmek pahalı.
- Twitter diyalogları, WhatsApp diyalogları, film / dizi diyalogları.
- Derlemde (D) kullanıcının söylediğine (q) en benzer ifadeyi (t') bul:

$$t' = \arg\max_{t \in D} \frac{q^T t}{\|q\|\|t\|}$$

- Cevap = t''ye cevap olarak verilen ifade.
- Derlem olarak diyaloglar yerine kitap, ansiklopedi vb. de kullanılabilir.
- Sorulara SSS'lerden cevap bulmak da benzer.

Yani elinizde soru-cevap ikilileri ve bir soru vardır. Sorunun elinizdeki soruların hangisine en çok benzediğini (kosinüs ile) bulur, onun cevabını kullanıcıya söylersiniz. Bunu sıkça sorulan sorulara cevap vermek gibi düşünebilirsiniz.

11.2 Soft cosine

Kosinüs benzerliğini one-hot gibi bir yapıyla hesaplarsanız kelimelerin anlamsal yakınlıklarına dair hiçbir şey çıkmaz (01:15:41). "Cimbom kazandı" ile "Galatasaray yendi" aynı şeyi söyler, ama one-hot kodlama kullanırsanız kelime bazında hiçbir benzerlik bulamazsınız. Soft cosine bunu biraz çözmek için kullanılır (ekrandan, 01:16:15):

$$\cos(X, Y) = \frac{X^t Y}{\sqrt{X^t X}\sqrt{Y^t Y}}, \qquad X^t Y = \sum_{i=1}^{n} x_i y_i$$

$$\cos_M(X, Y) = \frac{X^t M Y}{\sqrt{X^t M X}\sqrt{Y^t M Y}}, \qquad X^t M Y = \sum_{i=1}^{n}\sum_{j=1}^{n} x_i m_{i,j} y_j$$

X ile Y arasındaki benzerliği doğrudan bulmak yerine araya bir M benzerlik matrisi eklenir. M için slaytta iki yöntem vardır:

$$m_{i,j} = \max(0, \text{cosine}(v_i, v_j))^2 \quad (v : \text{kelimelerin word2vec vektörü})$$

$$m_{i,j} = \alpha \left(1 - \frac{\text{Levenshtein}(w_i, w_j)}{\max(\|w_i\|, \|w_j\|)}\right)^{\beta} \quad (\text{kelimeler arası edit distance})$$

Birinci yöntem kelime gömmeleriyle anlamsal benzerliği hesaba katar. İkincisi Levenshtein uzaklığıdır: bir kelimenin ötekine kaç karakter işlemiyle dönüştürülebildiğini hesaplar. Bu da yanlış yazım ya da hafifçe farklı yazım gibi durumları benzerliğin içine katar. Böylece M hem anlamsal (semantik) yapıları hem de sözdizimsel (sentaktik) varyasyonları kapsayabilir. Slayttaki kaynak: aclweb.org/anthology/S17-2003.

11.3 Hafızasız sistemler

“Bu sorunun cevabı şudur” biçimindeki bu yapıların problemi hafızasız olmalarıdır (01:17:37): kullanıcının bir önce sorduğu soru ile şimdi sorduğu soru birbirinden tamamen bağımsız kabul edilir. Bu, sıkça sorulan sorular içinde arama gibi durumlar için sorun değildir, çünkü sorular kabaca birbirinden bağımsızdır. Ama kullanıcıyla bir sohbete, bir diyaloğa girecekseniz durum farklıdır.

12. RNN’lerle Diyalog Öğrenme ve Microsoft Tay

12.1 Tarihçesi olan mimariler

Diyalog tarihçesi olan bir şeydir (01:18:11). Bir dizi vardır: kullanıcı bir şey der, siz bir şey dersiniz, kullanıcı yine bir şey der. Sizin söyleyeceğiniz şey yalnız kullanıcının son söylediğine değil, kullanıcının daha önce söylediklerine ve sizin onlara verdiğiniz cevaplara, hepsine birden bağlıdır. Genelde hayal ettiğimiz de budur. Bu durumda tarihçesi olan mimarilere ihtiyaç vardır; bilinen örnekleri RNN’ler, onların sürümleri ve LSTM’lerdir.

Hoca buradaki belleğin Memory Networks’teki bellek olmadığını, RNN’in kendi belleği olduğunu vurguladı (01:19:17): sürekli güncellenen bir durum (state). RNN’lerin klasik yapısında mevcut girişlere göre bir çıkış üretilir ve bu çıkış bir sonraki zamanda kendine girdi olarak verilir. Bu da mimarinin belleğe sahip olmasını sağlar.

12.2 Kodlayıcı-çözücü

Slayttaki “RNN’lerle diyalog öğrenme” (ekrandan, 01:20:00): yine diyalog derlemleri kullanılır. Örnek, 3 cümlelik bir diyaloğun öğrenimidir:

```
Mom, i don't feel so good
What's wrong?
I feel I'm going to pass out
```

Şemada her konuşma sırası (utterance) kelime kelime bir kodlayıcı (encoder hidden state) ile okunur ve bir konuşma temsiline (utterance representation) çevrilir. Bu temsil bağlam gizli durumuna (context hidden state) aktarılır; bağlam durumu konuşma boyunca ilerler. Çözücü (decoder), bağlam durumundan başlayarak bir sonraki konuşma sırasını kelime kelime tahmin eder (“what’s wrong ? ”, sonra “i feel like i’m going to pass out . ”). Kaynak: arXiv:1507.04808.

Bu bir kodlayıcı-çözücü (encoder-decoder) yapısıdır ve günümüzde de çok standart bir yaklaşımdır (01:20:19). Girdiyi bir vektöre dönüştüren kısma kodlayıcı, o vektörden üretilmek istenen şeye (metin ya da başka bir şey) dönüşüm yapan kısma çözücü denir. Genel olarak herhangi bir veri tipinden vektöre dönüşüme kodlayıcı, bir vektörden herhangi bir veri tipine (ses, görüntü, metin, video) dönüşüme çözücü denir.

12.3 Microsoft Tay

Slayt (ekrandan, 01:21:40):

- Kullanıcılarla yaptığı diyaloglardan da öğrenen bir sistem (derlemine onları da ekliyor).
- Açıldıktan 16 saat sonra kapatıldı.
- Neden?

Fikir şuydu: diyalogları kullanıcılardan toplarız, ama sistemin o anda kullanıcıyla geçen diyaloğu da hemen eğitim kümesine katması ve onu da öğrenmesi çok iyi olur. Bunu yaptıklarında sistem çok hızlı güncellendi ve işler çığırından çıktı (01:22:20): kötü niyetli kullanıcıların söylemleri botun söylemlerine temel oluşturdu, kullanıcılar ne söylüyorsa bot da onları söylemeye başladı ve sistem kapatıldı.

Hoca bunun bayağı bir ders olduğunu söyledi: hiçbir şey bir anda ilerlemez; hatalardan çıkarılan derslerle gide gide daha başarılı sistemler oluşur. ChatGPT özelinde, sizin söyledikleriniz anında veri tabanına girip kullanılmaz; hepsi belli filtrelerden geçer (01:23:10). Çok büyük miktarda veri olduğu için istenmeyen türden veriler içeri katılmaz.

Ek bilgi (derste anlatılmadı): Microsoft Tay büyük olasılıkla Mart 2016'da Twitter üzerinde yayına alınmış bir sohbet botudur. Bazı kullanıcıların bilinçli olarak saldırgan ve ırkçı ifadeler öğretmesiyle bot benzer ifadeler üretmeye başlamış ve kısa sürede kapatılmıştır.

13. Sanal Asistanlar

13.1 Ticari diyalog sistemleri

Sohbetten sonra sanal asistanlara geçildi; bu, diyalog sistemlerinin ticari tarafıdır (01:23:46). Slayttaki "Sanal Asistanlar" (ekrandan, 01:24:10):

- Goal oriented, task based, frame based (hedef odaklı, görev tabanlı, çerçeve tabanlı).
- Genel yapı 1977'den beri değişmedi: GUS.
- Bir uygulama (takvim yönetimi, bilet bulma, öneride bulunma vb.) için yardımcı sistemler.
- Yapılabilecek işlemlerin türleri belirlenir.
- Her işlem türü için gerekli bilgiler (slot) ve türleri (zaman, sayı, şehir vb.) belirlenir.
- Bu bilgileri almak için sorular belirlenir.
- Bir sıra ile bilgiler alınır.

Kaynak: web.stanford.edu/~jurafsky/slp3/slides/convagents1.pdf.

13.2 Neden yavaş ilerliyor: güvenlik

Hoca, bir bankanın chatbotunun ChatGPT gibi sistemlerle entegrasyonunun o kadar hızlı ilerlemediğine dikkat çekti (01:24:50). Sebep güvenliktir (safety). ChatGPT ile sohbet ederken model yanlış bir şey yaptığında gülersiniz ("abi bu ne"); iyi bir şey yaptığında şaşırırsınız. Ticari tarafta ise bir hedefiniz vardır: birine para göndereceksiniz ya da akıllı bir yatırım yapmaya çalışıyorsunuz. "Yanlış yap da güleyim" pozisyonunda olmazsınız; "%5 para kaybettirdin, ne komik" demezsiniz.

Bu yüzden bireysel kullanıcılara yönelik tarafta çok iyi işler çıkarken ticari tarafta uygulamalar o kadar hızlı ilerlemez, çünkü hata kaldırmaz (01:26:00). %99 başarıyla çalışan bir sistemi bile ticari tarafta kimse kabul etmeyebilir. Bu yüzden ticari taraftaki yapılar daha geleneksel ve katıdır (rigid): "yapamıyorsa o işi yapmasın, ama yanlış bir şey yapmasın; gerekirse orada insan çalıştırırız."

Bu yapılar ELIZA ve PARRY'deki gibi çok fazla emekle oluşturulur: kullanıcılar neler ister, bu istekleri nasıl alırız, hangi soruları sorarız, kullanıcı ne dediğinde ne anlamalıyız? Belli parçalarda makine öğrenmesi teknikleri entegre edilir, ama diyaloğun tamamını üretici modellere yaptırmak henüz çok uygulanabilir görünmemektedir. Hocaya göre ileride ne olacağını göreceğiz, ama bu sistemlerde güvenliğin yüksek olması gerekir. Web aramasında kullanıcının karşısına yanlış bilgi çıkması olabilir; kullanıcı bunu fark edebilir, fark etmese bile sonuç korkunç olmayabilir ve "burada gördüğünüz her şeye inanmayın" diye bir uyarı koyabilirsiniz. Ama bir bankanın chatbotunun üzerine "bu sistemin söylediği her şeye inanmayın" yazamazsınız (01:28:26).

13.3 Slot doldurma örneği

Bu sistemlerde kullanıcının söylediğinden kastettiği şeyi anlamak ve işinizi yapmanız için gereken hangi bilgileri henüz almadığınızı fark edip onları almaya çalışmak gibi teknikler vardır (01:29:08). Slayttaki örnek (ekrandan, 01:29:20):

```
Ankara'ya giden uçuşlar hangileri?
  - Niyet: Uçuş sorgulama
  - Kalkış şehri: Kullanıcının şehri -> İstanbul
  - Kalkış tarihi: ??
  - Kalkış zamanı: ??
  - Varış şehri: Ankara
Kalan boşlukları doldurmak için soru sor:
  - hangi tarih için uçuş arıyorsunuz?
  - İstanbul'dan Ankara'ya hangi tarih için uçuş arıyorsunuz?
```

İkinci soruda "İstanbul'dan Ankara'ya" kısmı slaytta kırmızıyla vurgulanmıştır. Hoca, kullanıcının söylediği şeyin anlaşılıp anlaşılmadığını ona hissettirmenin çok kritik olduğunu söyledi (01:29:56). Restoran öneren sistemler de benzer örneklerdir. Bunların hepsinin ortak özelliği, işi mümkün olduğunca elle tasarlanmış adımlarla ilerletmeleridir.

Örnek Sorular

Soru 1 (ekrandan, 00:24:45): QANet slaytında $A = \bar{S} \cdot Q^T$ ve $B = \bar{S} \cdot \bar{\bar{S}}^T \cdot C^T$ tanımlanıyor (S: $n \times m$ benzerlik matrisi, $\bar{S}$ satır, $\bar{\bar{S}}$ sütun softmax normalizasyonu, Q: $d \times m$, C: $d \times n$). A ve B neyi ifade eder?

> **Çözüm (derste verilmedi):** Boyutlar: $\bar{S}$ $n \times m$, Q^T $m \times d$ olduğundan A $n \times d$'dir; $\bar{S}\bar{\bar{S}}^T$ $n \times n$, C^T $n \times d$ olduğundan B de $n \times d$'dir (python ile doğrulandı; $[C; A; C \odot A; C \odot B]$ $4d \times n$ çıkar). A'nın i. satırı, paragrafın i. kelimesinin soru kelimelerine verdiği dikkat ağırlıklarıyla soru kelime vektörlerinin ağırlıklı ortalamasıdır: her paragraf kelimesi için "sorunun kendisiyle ilgili kısmı" (bağlamdan soruya dikkat, context-to-query attention). B önce her paragraf kelimesini soru kelimeleri üzerinden yine paragraf kelimelerine bağlar, sonra paragraf vektörlerinin ağırlıklı ortalamasını alır: her paragraf kelimesi için soruya göre önemli olan paragraf kısımlarının özeti (sorudan bağlama dikkat, query-to-context attention). Bu yorum büyük olasılıkla QANet makalesindeki (ve ondan önceki BiDAF'taki) tanıma karşılık gelir.

Soru 2 (ekrandan, 01:21:40): Microsoft Tay, kullanıcılarla yaptığı diyaloglardan da öğrenen (derlemine onları da ekleyen) bir sistemdi ve açıldıktan 16 saat sonra kapatıldı. Neden?

Cevap (derste verildi): Kullanıcılarla geçen diyaloglar filtrelenmeden, hemen eğitim kümesine katıldı. Kötü niyetli kullanıcıların söylemleri botun söylemlerine temel oluşturdu ve bot kullanıcıların ona söylediklerini tekrar etmeye başladı. Bu yüzden ChatGPT gibi sistemlerde kullanıcı girdileri anında eğitime girmez, filtrelerden geçer.

Soru 3 (derste sözlü, 00:37:26): Bir sonraki token tahmini (next token prediction) ne tür bir problemdir?

Cevap (derste verildi): Bir sınıflandırma problemidir, regresyon değildir. Girdiye karşılık sınıflardan birinin indisi tahmin edilir; sınıflar karakter düzeyinde 84 karakter, tokenizer kullanılırsa 32 bin ya da 48 bin token gibi bir sözlüktür. Farkı yaratan, öz denetimli olarak insan emeği olmadan çok basit bir kodla üretilebilen çok büyük eğitim kümesidir.

Soru 4 (derste sözlü, 00:34:54): Bir sonraki token tahmini neden bu kadar etkilidir?

Cevap (derste verildi): Hipotez şudur: bir dizinin devamını tahmin edebiliyorsanız o diziyi üreten süreci büyük ölçüde kavramışsınızdır. Ancak "kavramak" insan anlamında alınmamalıdır; bir problemi çözebilmek onu istediğimiz yoldan çözmek demek değildir.

Soru 5 (derste sözlü, 01:05:11): Aynı LSTM mimarisiyle ve benzer büyüklükte veriyle üretilen şiirler, üretilen spor haberlerinden neden daha başarılı görünür?

Cevap (derste verildi): Birincisi, şiir hata kaldırır: garip bir ifade ("Bahçem yaralı kapılar gibi") sembolik olarak yorumlanabilir, düz metindeki hata ise hemen göze batar. İkincisi, şiirlerin çeşitliliği çok yüksektir; spor haberleri belli bir kalıba sahiptir ve az çeşitlidir. Mimari ve veri büyüklüğü açısından fark yoktur.

Soru 6 (ekrandan, 01:29:20): "Ankara'ya giden uçuşlar hangileri?" isteğinde hangi slotlar doludur, hangileri boştur ve boşlukları doldurmak için nasıl soru sorulur?

Cevap (derste verildi, slayttan): Niyet uçuş sorgulama; kalkış şehri kullanıcının şehrinden İstanbul; varış şehri Ankara. Kalkış tarihi ve kalkış zamanı boştur. Boşluk için "hangi tarih için uçuş arıyorsunuz?" ya da, kullanıcıya anlaşıldığını hissettiren biçimiyle, "İstanbul'dan Ankara'ya hangi tarih için uçuş arıyorsunuz?" diye sorulur.

Dersin Toparlaması

Ders, bellek tabanlı soru cevaplama modellerinden başladı. Memory Networks bellekteki ilgili cümleleri argmax ile seçip modeli destekleyici cümle indisleriyle yönlendirirken, End-to-End Memory Networks bu indislere gerek duymadan yumuşak ağırlıklar, iki ayrı bellek temsili, çok adımlı yapı ve konum gömmeleri kullandı; hoca bunları Transformer ve BERT'e giden adımlar olarak yorumladı. Bu modellerin başarısına yöneltilen itiraz, bir veri kümesindeki başarının o işin çözüldüğü anlamına gelmediğini, modellerin istemediğimiz kısa yollar bulabildiğini gösterdi. QANet ile paragrafta cevabın başlangıç ve bitişini seçen yaklaşım görüldü ve buradan ChatGPT ile öne çıkan üretici yaklaşıma geçildi.

Dersin ikinci yarısı üretime ve diyaloğa ayrıldı. Otoregresif üretimin aslında bir sonraki token için bir sınıflandırma olduğu, gücünü öz denetimli büyük veriden aldığı vurgulandı; küçük LSTM'lerle şiir ve haber üretimi örnekleri lokal iyiliğin kolay, bütünlüğün zor olduğunu, değerlendirmenin ise üretici modellerin en büyük problemi olduğunu gösterdi. Diyalog sistemlerinde ELIZA ve PARRY gibi şablon tabanlı yapılardan derlem ve soft cosine tabanlı geri getirmeye, oradan RNN kodlayıcı-çözücülere ve Microsoft Tay'in dersine ulaşıldı. Son olarak ticari sanal asistanların güvenlik gereği hâlâ GUS'tan beri değişmeyen çerçeve ve slot tabanlı katı yapılarla ilerlediği anlatıldı. Hoca bir sonraki derste görsel anlambilime de girileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.